

minimum digits

(smallest no. will always have digits in ascending order.)

arr = [$\overset{\checkmark}{2}_0$, $\overset{\checkmark}{3}_1$, $\overset{\checkmark}{0}_2$, $\overset{\checkmark}{9}_3$, $\overset{\checkmark}{0}_4$, $\overset{\checkmark}{8}_5$]

num1 = 028

num2 = 039

67

dry run

2	9
3	0
0	8

minHeap

mem = ~~0~~ ~~0~~ ~~2~~ ~~3~~ ~~8~~ 9

num1 = 28

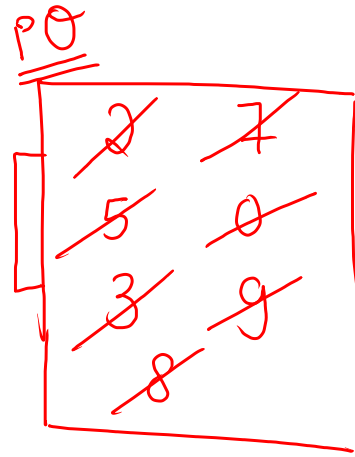
num2 = 39

code

```
public static void main(String[] args) {
    Scanner scn = new Scanner(System.in);
    int n = scn.nextInt();
    int[] arr = new int[n];
    for (int i = 0; i < n; i++) {
        arr[i] = scn.nextInt();
    }
    System.out.println(minimumDigits(arr, n));
}

public static long minimumDigits(int[] arr, int n) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for (int i : arr) {
        pq.add(i);
    }

    → long num1 = 0;
    → long num2 = 0;
    while (pq.size() > 0) {
        → int rem = pq.poll();
        if (pq.size() % 2 == 0) {
            num1 = num1 * 10 + rem;
        } else {
            num2 = num2 * 10 + rem;
        }
    }
    return num1 + num2;
}
```



Size = ~~7~~ ~~5~~ ~~4~~ ~~3~~ ~~2~~ ~~1~~ ~~0~~

rem = ~~0~~ ~~2~~ ~~3~~ ~~5~~ ~~7~~

~~8~~ ~~9~~

num1 = 379

num2 = 258

Minimum Cost of ropes 3

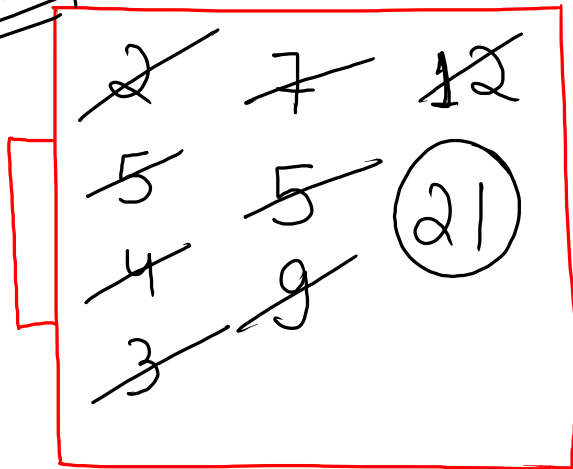
$$\text{arr} = [\overset{0}{\cancel{2}}, \overset{1}{\cancel{5}}, \overset{2}{\cancel{4}}, \overset{3}{\cancel{3}}, \overset{4}{\cancel{7}}] \cancel{5}, \cancel{9}, \cancel{12} \quad 21$$

$$\text{cost} = 7 + 11 + 14 + 21 = 53$$

$$\text{cost} = 5 + 9 + 12 + 21 = 47$$

arr = [⁰2, ¹5, ²4, ³3, ⁴7]

minHeap



$$\text{cost} = 0 + 5 + 9 + 12 + 21$$

Note:- after combining 2 ropes, make sure to add that rope in PO again

code

```
public static void main(String[] args) {
    Scanner scn = new Scanner(System.in);
    int n = scn.nextInt();
    int[] arr = new int[n];
    for (int i = 0; i < n; i++) {
        arr[i] = scn.nextInt();
    }
    System.out.println(miniCostOfRopes(arr, n));
}

public static int miniCostOfRopes(int[] arr, int n) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    [ for (int i : arr)
        pq.add(i);

    int cost = 0;
    [ while ( pq.size() > 1 ) {
        int rope1 = pq.poll();
        int rope2 = pq.poll();
        cost += rope1 + rope2;
        pq.add( rope1 + rope2 );
    }
    return cost;
}
```

T.C = $O(n \log n)$

subtract numbers 1

$$\text{arr} = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ [1, & 5, & 0, & 3, & 5, & 5, & 1, & 3] \end{matrix}$$

$$\underline{\underline{x=1}}, \text{arr} = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ [0, & 4, & 0, & 2, & 4, & 4, & 0, & 2] \end{matrix} \checkmark$$

$$\underline{\underline{x=2}}, \text{arr} = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ [0, & 2, & 0, & 0, & 2, & 2, & 0, & 0] \end{matrix} \checkmark$$

$$\underline{\underline{x=2}}, \text{arr} = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ [0, & 0, & 0, & 0, & 0, & 0, & 0, & 0] \end{matrix} \checkmark$$

ans = no. of operations = 3

Ans :- no. of non-zero unique
elements

hashset

1
5
3

set.size() = 3

Code

```
public static void main(String[] args) {
    Scanner scn = new Scanner(System.in);
    int n = scn.nextInt();
    int[] arr = new int[n];
    for (int i = 0; i < n; i++) {
        arr[i] = scn.nextInt();
    }
    System.out.println(subtractNum(arr, n));
}

public static int subtractNum(int[] arr, int n) {
    HashSet<Integer> set = new HashSet<>();
    for (int i : arr)
        if (i > 0)
            set.add(i);
    return set.size();
}
```

Revision:-

- ✓ → sorting, lambda function
- ✓ → arrays, subarrays (Kadan's algo)
- 2 pointers
- Prefix array
- Arrays as Hashmap
- 2d arrays
- Strings, substring
- Binary Search (BSLB, BSUB)
- Arraylist
- Stack
- Hashmap, HashSet
- Queue
- PO

→ Sorting

{ Bubble sort
Insertion sort
Selection sort

{ Arrays.sort(arr)
Arrays.sort(arr, Collections.reverseOrder())

Arrays.sort(arr, (a, b) → {

return —————→ a - b // ↑ing order based on value
});
 └──→ b - a // ↓ing " " "
 └──→ -1 // a will print first
 └──→ +1 // b " " "

form the largest no.

Sample Input 0

4
4 46 8 9

→ convert to string

→ lambda fⁿ logic :-

str1 = a+b
str2 = b+a

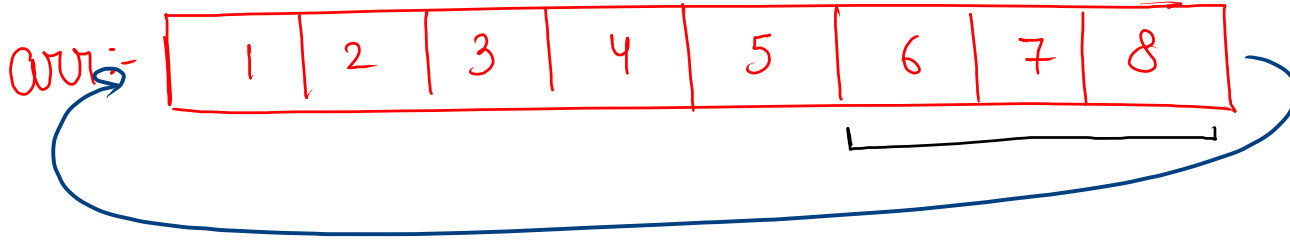
Sample Output 0

→ print ans

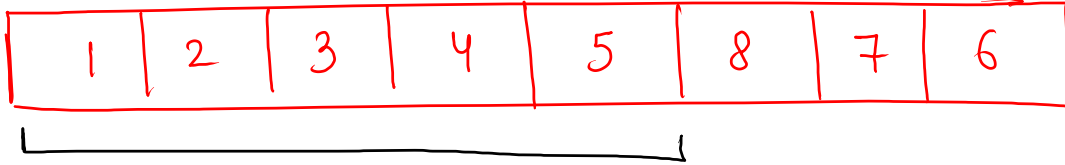
Sort Array By Parity

- $a = \text{even}, b = \text{odd}, \text{return } -1$
- $a = \text{odd}, b = \text{even}, \text{return } +1$
- $a = \text{even}, b = \text{even}, \text{return } a - b$
- $a = \text{odd}, b = \text{odd}, \text{return } a - b$

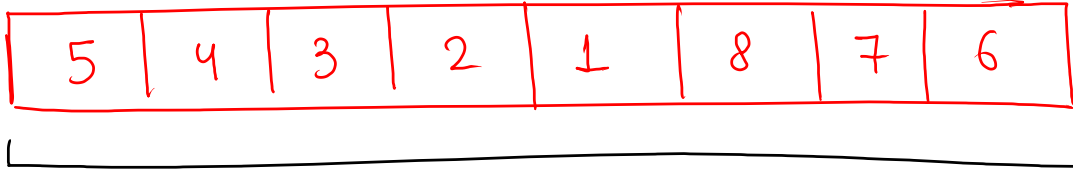
⇒ Rotate Right (k=3)



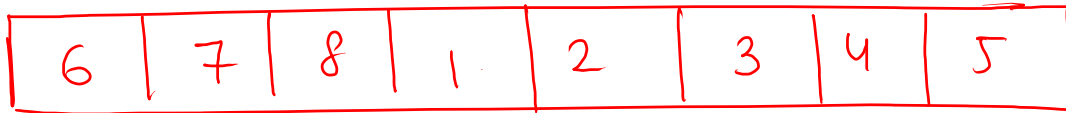
step 1



step 2



step 3



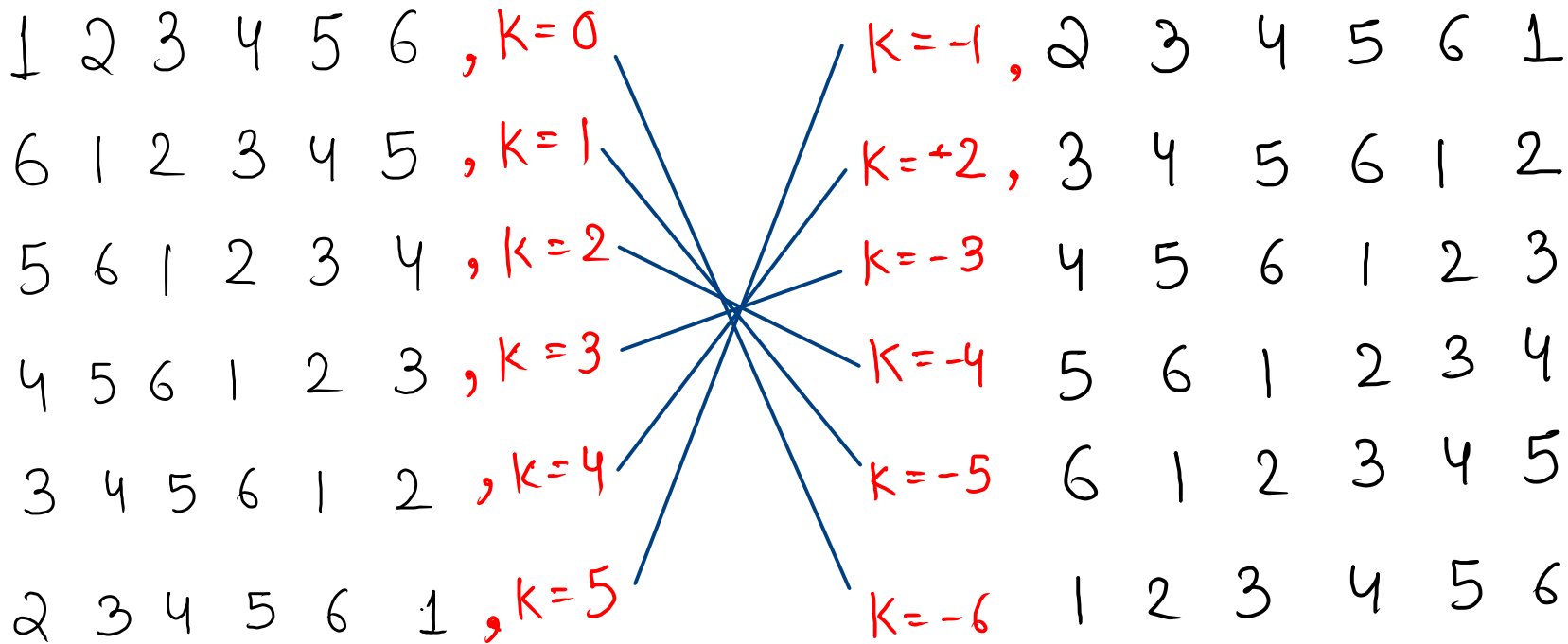
observation

1 2 3 4 5 6	,	$K=0, 6, 12 \dots$
6 1 2 3 4 5	,	$K=1, 7, 13 \dots$
5 6 1 2 3 4	,	$K=2, 8, \boxed{14} \dots$
4 5 6 1 2 3	,	$K=3, 9, 15 \dots$
3 4 5 6 1 2	,	$K=4, 10, 16 \dots$
2 3 4 5 6 1	,	$K=5, 11, 17 \dots$

$$\underline{\underline{K=14}}$$

$$\boxed{K = K \% n}$$
$$= 14 \% 6$$
$$= 2$$

observation



$$k = k + n$$

$$\begin{aligned} k &= -4 \\ k &= -4 + 6 \\ &= 2 \end{aligned}$$

Max Subarray 2

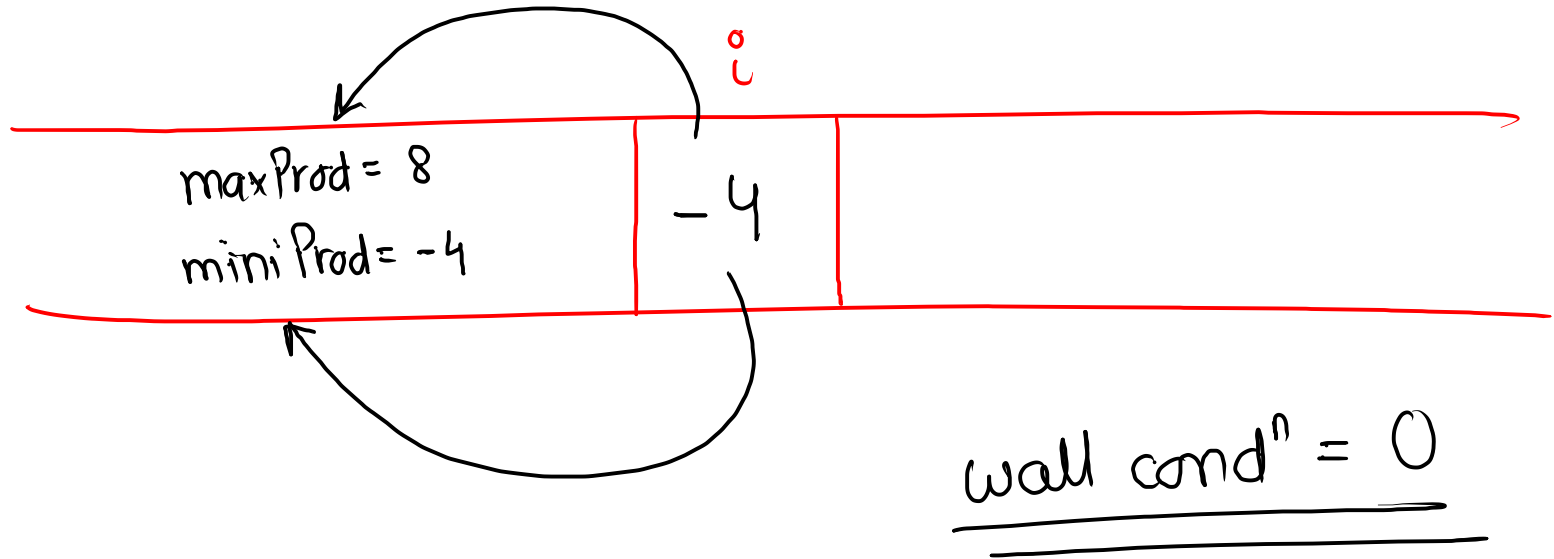
arr = [⁰2, ¹4, ²-7, ³6, ⁴2, ⁵-18, ⁶1, ⁷2, ⁸2, ⁹-6]

↑
i

```
public static int maxSumSubarray(int[] arr, int n) {  
    int sumSoFar = 0;  
    int maxSum = Integer.MIN_VALUE;  
    for (int i = 0; i < n; i++) {  
        if ( sumSoFar < 0 ) {  
            sumSoFar = arr[i];  
        } else {  
            sumSoFar += arr[i];  
        }  
  
        if ( sumSoFar > maxSum ) {  
            maxSum = sumSoFar;  
        }  
    }  
    return maxSum;  
}
```

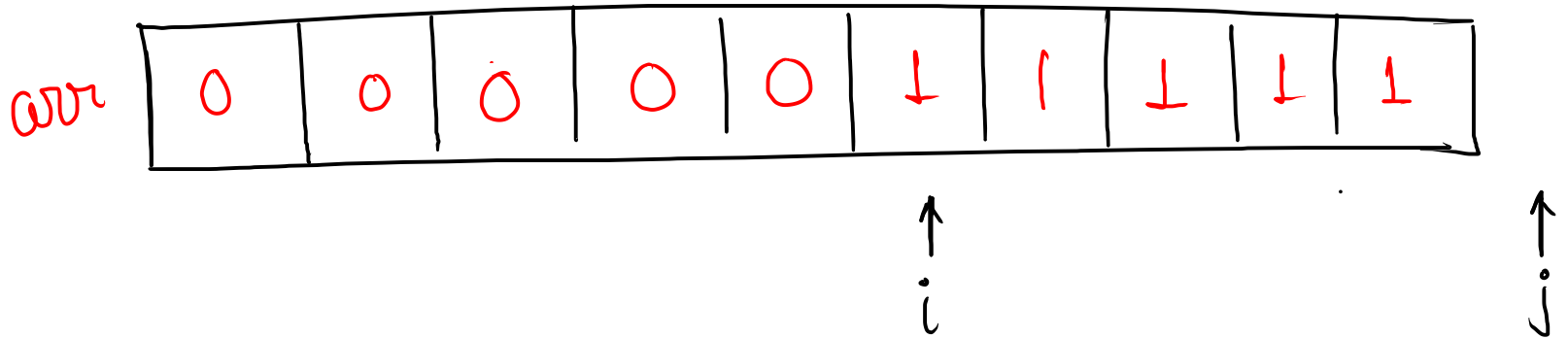
sumSoFar = ~~0~~ ~~2~~ ~~6~~ ~~-1~~ ~~6~~ ~~8~~ ~~-10~~ ~~1~~ ~~3~~ ~~5~~ ~~-1~~
maxSum = ~~-6~~ ~~2~~ ~~6~~ 8

Maximum Product Subarray 2

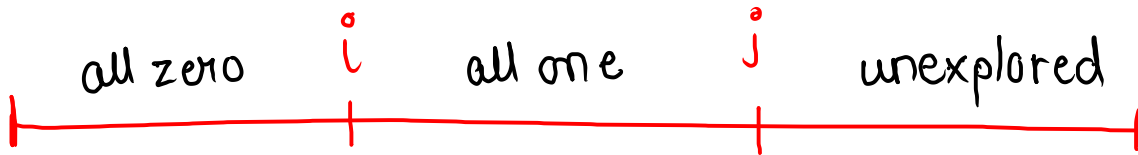


$$\text{maxProd} = \max(\text{curr} * \text{maxProd}, \text{curr} * \text{minProd}, \text{curr});$$
$$\text{minProd} = \min(\text{curr} * \text{maxProd}, \text{curr} * \text{minProd}, \text{curr});$$

⇒ 2 pointer



faith:-



in $O(n)$ time

$\Rightarrow 3 \text{ sum}$

$$\text{arr}[i] + \text{arr}[j] + \text{arr}[k] = 0$$

$$\underline{\underline{\text{arr}[i] + \text{arr}[j]}} = \underline{\underline{-1 * \text{arr}[k]}}$$



target

target sum for each k value